

Library Management System

MongoDB Course

By - Veer Pratap Singh

23BCE1340

Technology Stack : Node.js (Express) + MongoDB

API Testing : curl / Postman

1. Introduction

The Library Management System is a backend REST API application built with Node.js and the Express framework, backed by a MongoDB NoSQL database that stores and manages all library data. Object modeling and validation are handled through Mongoose. Alongside the API, the project ships a lightweight web panel (plain HTML, CSS, and JavaScript) so the same endpoints can be exercised either from a browser, from curl, or from Postman.

The project demonstrates the core ideas of backend development: CRUD operations (Create, Read, Update, Delete), REST API design, MongoDB document modeling, references between collections using ObjectId, and token-based authentication with role separation. A copy of every request can be reproduced with the bundled `api-requests.sh` file, and the whole stack runs with a single Docker Compose command.

2. Objective

- Design and implement a complete REST API for a library system using Express and Mongoose.
- Perform full CRUD operations on Books, Members, and Loans.
- Model MongoDB relationships (One-to-Many) between members, books, and loan records using ObjectId references.
- Keep book availability consistent automatically whenever a book is issued, borrowed, or returned.
- Secure the API with JWT authentication and two roles - *member* and *admin* - so that members can only act on their own loans.
- Provide a browser-based web panel in addition to the raw API.
- Verify every endpoint with curl and Postman, and package the application with Docker Compose.

3. Project Scope

The system manages three core entities: Books, Members, and Loans. The Loan collection records which member borrowed which book, the issue date, the due date, the return date, and the current status. Members authenticate with an email and password; an admin manages the catalogue and issues books, while a regular member browses books, borrows for themselves, and views or returns only their own loans. Books can be searched by title, author, or ISBN, and filtered by availability. The project intentionally stays at MVP scope - features such as fines, reservations, and pagination are left out so the focus remains on MongoDB modeling and clean API design.

4. Technology Stack

Technology	Version	Purpose
Node.js	18+ (tested on 22)	JavaScript runtime for the server

Express	4.19	REST API / HTTP framework
MongoDB Community	7	NoSQL document database
Mongoose	8.5	ODM - schemas, validation, query building
bcryptjs	3.0	One-way password hashing
jsonwebtoken	9.0	JWT generation and verification (auth)
dotenv	16.4	Loads configuration from .env
cors	2.8	Cross-origin resource sharing
Docker + Docker Compose	Latest	Containerized app + MongoDB deployment
nodemon	3.1	Auto-reload during development
HTML / CSS / Vanilla JS	-	Browser web panel frontend
curl / Postman	Latest	API testing and verification

5. Project Structure

The codebase follows a modular, layered structure: routes define the endpoints, controllers hold the request handling and business logic, models declare the Mongoose schemas, and middleware handles auth and errors. The frontend lives separately under `public/`.

```

libman/
  src/
    server.js          -- boot: connect to MongoDB, start HTTP server
    app.js             -- Express app: middleware + route mounting
    config/db.js       -- Mongoose connection helper
    models/            -- Mongoose schemas: Book, Member, Loan
    controllers/       -- CRUD + auth + loan business logic
    routes/            -- Express routers (auth, books, members, loans)
    middleware/        -- auth (JWT), asyncHandler, error handler
    utils/token.js     -- JWT sign / verify helpers
    seed/seed.js       -- inserts sample books and members
  public/              -- web panel (index.html, css, js)
  api-requests.sh      -- copyable curl reference for every endpoint
  Dockerfile           -- app container image
  docker-compose.yml   -- app + MongoDB + one-shot seed
  .env.example         -- configuration template

```

6. MongoDB Collections and Relationships

MongoDB is a NoSQL document database. Instead of tables and rows it stores collections of documents in a JSON-like binary format (BSON). Relationships are expressed by storing the ObjectId of one document as a reference field inside another. This project uses three collections.

Collection	Key Fields	Description
books	title, author, isbn (unique), category, publishedYear, totalCopies, availableCopies, description	Stores book catalogue and live stock counts.
members	name, email (unique), phone, membershipId (unique), role, passwordHash, status	Stores members and admins. role is "member" or "admin"; the hashed password is never returned.
loans	book (ref), member (ref), issueDate, dueDate, returnDate, status	One record per issue/return. book references books, member references members.

Relationships Demonstrated

- **One-to-Many:** one Member can have many Loans (the member ObjectId is stored in each loan).
- **One-to-Many:** one Book can appear in many Loans over time (the book ObjectId is stored in each loan).
- **Many-to-One:** many Loans reference exactly one Book and one Member.
- On read, loans are populated so each record returns the linked book and member details in a single response.

When a book is issued or borrowed, the `availableCopies` field on the book is decreased; when it is returned, the count is increased again. The decrement is done with a single atomic conditional update (`findOneAndUpdate` guarded by `availableCopies > 0`), so two simultaneous requests can never both claim the last copy. This keeps stock consistent across collections in real time without requiring multi-document transactions.

Authentication and Roles

On login or registration the server returns a signed JWT containing the member id and role. Protected routes require an `Authorization: Bearer <token>` header. Read access to books is open to any logged-in user; creating/editing books and all member management require the admin role; a member may only list, view, or return their own loans.

7. API Endpoints

The system exposes 19 REST endpoints across 5 groups. Each uses the appropriate HTTP method and returns a JSON response with a meaningful status code. All routes except Auth and the health check require a Bearer token.

Auth

Method	Endpoint	Status	Description
POST	/api/auth/register	201 Created	Register a member, returns a JWT
POST	/api/auth/login	200 OK	Log in, returns a JWT
GET	/api/auth/me	200 OK	Current logged-in user

Books (read: any user | write: admin)

Method	Endpoint	Status	Description
POST	/api/books	201 Created	Add a new book (admin)
GET	/api/books	200 OK	List / search (?search, ?available)
GET	/api/books/:id	200 OK	Retrieve one book
PUT	/api/books/:id	200 OK	Update a book (admin)
DELETE	/api/books/:id	200 OK	Delete a book (admin, blocked if on loan)

Members (admin only)

Method	Endpoint	Status	Description
POST	/api/members	201 Created	Create a member
GET	/api/members	200 OK	List / search (?search, ?status)
GET	/api/members/:id	200 OK	Retrieve one member
PUT	/api/members/:id	200 OK	Update a member
DELETE	/api/members/:id	200 OK	Delete (blocked if active loan)

Loans

Method	Endpoint	Status	Description
POST	/api/loans	201 Created	Admin issues a book to a member

POST	/api/loans/borrow	201 Created	Member borrows a book for themselves
GET	/api/loans	200 OK	List loans (member: own only; admin: all, ?status/?member/?book)
GET	/api/loans/:id	200 OK	Retrieve one loan (owner or admin)
POST	/api/loans/:id/return	200 OK	Return a borrowed book

System

Method	Endpoint	Status	Description
GET	/api/health	200 OK	Service health check (no auth)

8. API Testing Using curl and Postman

Every endpoint was tested against the running server at `http://localhost:3000` with MongoDB active on port 27017. The captures below show the actual request sent and the JSON response returned by the server. (The JWT token is shortened for readability.)

Figure 1: `POST /api/auth/login` - authenticate as admin (HTTP 200 OK, returns a JWT)

```
$ curl -X POST http://localhost:3000/api/auth/login -H "Content-Type: application/json" \
-d '{"email":"admin@example.com","password":"admin123"}'

{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMzU5Ni...<shortened>",
  "data": {
    "id": "6a35961d555cafea4427d7dc",
    "name": "Admin",
    "email": "admin@example.com",
    "role": "admin",
    "membershipId": "LIB-MQLBB3B0-6931"
  }
}
```

Login verifies the email and bcrypt-hashed password, then returns a signed JWT plus the user's public details. This token is sent as a Bearer header on every protected request that follows.

Figure 2: `GET /api/books` - retrieve all books (HTTP 200 OK, count = 5)

```
$ curl http://localhost:3000/api/books -H "Authorization: Bearer <ADMIN_TOKEN>"

{
  "success": true,
  "count": 5,
  "data": [
    {
      "_id": "6a35961d555cafea4427d7d6",
      "title": "The Pragmatic Programmer",
      "author": "Andrew Hunt",
      "isbn": "978-0135957059",
      "category": "Programming",
      "totalCopies": 3,
      "availableCopies": 2
    },
    { "...": "4 more book documents" }
  ]
}
```

The endpoint returns every document in the books collection together with a total count. Each book carries its stock counts, so `availableCopies` of 2 out of 3 indicates one copy is currently on loan.

Figure 3: `POST /api/books` - add a new book (HTTP 201 Created)

```
$ curl -X POST http://localhost:3000/api/books -H "Authorization: Bearer <ADMIN_TOKEN>" \
-H "Content-Type: application/json" \ -d '{"title":"Refactoring","author":"Martin Fowler","isbn":"978-0201485677","totalCopies":2}'
```

```
{
  "success": true,
  "data": {
    "title": "Refactoring",
    "author": "Martin Fowler",
    "isbn": "978-0201485677",
    "totalCopies": 2,
    "availableCopies": 2,
    "_id": "6a35961eb065d416f04bb8c8",
    "createdAt": "2026-06-19T19:18:54.499Z"
  }
}
```

A new document is inserted into the books collection. The server responds with HTTP 201 and the full document including the auto-generated MongoDB ObjectId; availableCopies defaults to totalCopies.

Figure 4: POST /api/loans - admin issues a book to a member (HTTP 201 Created)

```
$ curl -X POST http://localhost:3000/api/loans -H "Authorization: Bearer <ADMIN_TOKEN>" \
-H "Content-Type: application/json" -d '{"bookId":"6a35961eb065d416f04bb8c8","memberId":"6a35961d555cafea4427d7de"}'
```

```
{
  "success": true,
  "data": {
    "book": { "title": "Refactoring", "isbn": "978-0201485677" },
    "member": { "name": "Alice Johnson", "membershipId": "LIB-MQLBB3CW-6242" },
    "dueDate": "2026-07-03T19:18:54.585Z",
    "status": "issued",
    "_id": "6a35961eb065d416f04bb8d0",
    "isOverdue": false
  }
}
```

Issuing creates a loan that references the book and the member by ObjectId (returned here already populated). The book's availableCopies is atomically decremented in the same operation, and a 14-day due date is set by default.

Figure 5: POST /api/loans/borrow - member borrows for themselves (HTTP 201 Created)

```
$ curl -X POST http://localhost:3000/api/loans/borrow -H "Authorization: Bearer <MEMBER_TOKEN>" \
-H "Content-Type: application/json" -d '{"bookId":"6a35961eb065d416f04bb8c8"}'
```

```
{
  "success": true,
  "data": {
    "book": { "title": "Refactoring", "isbn": "978-0201485677" },
    "member": { "name": "Alice Johnson", "membershipId": "LIB-MQLBB3CW-6242" },
    "status": "issued",
    "_id": "6a35961eb065d416f04bb8d8",
    "isOverdue": false
  }
}
```

The self-service route takes only a book id - the member is read from the JWT, so a member can never borrow on someone else's behalf. This is the key difference from the admin issue route.

Figure 6: GET /api/loans with a member token - loans are scoped to the caller (count = 3)

```
$ curl http://localhost:3000/api/loans -H "Authorization: Bearer <MEMBER_TOKEN>"
```

```
{
  "success": true,
  "count": 3,
  "data": [
    {
      "book": { "title": "Refactoring" },
      "member": { "name": "Alice Johnson" },
      "status": "issued",
      "id": "6a35961eb065d416f04bb8d8"
    },
    { "...": "only this member's loans are returned" }
  ]
}
```

The same endpoint behaves differently per role: a member sees only their own loans (enforced server-side from the token), while an admin sees every loan and can filter by status, member, or book. This demonstrates authorization on top of authentication.

9. Conclusion

The Library Management System successfully demonstrates a complete backend REST API built with Node.js, Express, and MongoDB. The project covers the fundamentals expected of a backend service:

- **CRUD operations:** Create, Read, Update, and Delete implemented for Books and Members, with issue/return lifecycle for Loans.
- **MongoDB document modeling:** three collections of flexible JSON documents with schema validation via Mongoose.
- **Cross-collection relationships:** loans reference Books and Members by ObjectId and are populated on read.
- **Data integrity:** book availability is updated atomically on issue, borrow, and return, preventing over-issue of the last copy.
- **Authentication and authorization:** JWT login with bcrypt-hashed passwords and member/admin role separation.
- **REST API design:** correct HTTP methods and status codes with consistent JSON responses.
- **Modular structure:** routes, controllers, models, and middleware kept in separate layers.
- **Verification and deployment:** every endpoint exercised with curl/Postman; the stack runs from a single Docker Compose command.

Beyond the API, the project also includes a browser web panel that consumes the same endpoints, giving both a programmatic and a visual way to operate the library. The code follows clean, modular practices and can be extended later with features such as fines, reservations, or pagination.